Hotel Revenue Analytics

Selected code: feature engineering, probability modeling, and back-testing.

```python
"""Core functions from a hotel revenue-management project."""
from dataclasses import dataclass

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from scipy.stats import ttest_rel
from sklearn.compose import ColumnTransformer
from sklearn.ensemble import RandomForestClassifier
from sklearn.model_selection import train_test_split
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import OneHotEncoder


def add_revenue_metrics(bookings: pd.DataFrame,
                        weekly_adr: pd.DataFrame) -> pd.DataFrame:
    """Wrangle bookings and calculate revenue and opportunity costs."""
    data = bookings.copy()
    data["ArrivalDate"] = pd.to_datetime(data["ArrivalDate"])
    data["ReservationStatusDate"] = pd.to_datetime(data["ReservationStatusDate"])
    data["Nights"] = data.StaysInWeekendNights + data.StaysInWeekNights
    data["RevenuebyStays"] = data.Nights * data.AverageDailyRate

    assigned_adr = weekly_adr.rename(
        columns={"ReservedRoom": "AssignedRoom", "ADR": "AssignedADR"})
    data = data.merge(assigned_adr,
                      on=["ArrivalWeekNumber", "AssignedRoom"], how="left")
    upgraded = ((data.ReservedRoom != data.AssignedRoom)
                & data.ReservationStatus.eq("Check-Out"))
    opportunity_cost = (data.AssignedADR - data.AverageDailyRate) * data.Nights
    data["RevenueLossbyUpgrade"] = np.where(
        upgraded, opportunity_cost.clip(lower=0), 0)

    days_before = (data.ArrivalDate - data.ReservationStatusDate).dt.days
    late_cancel = (data.ReservationStatus.eq("Canceled")
                   & days_before.between(0, 3))
    cancel_cost = data.AverageDailyRate * (data.Nights - 1).clip(lower=0)
    data["RevenueLossbyCancel"] = np.where(late_cancel, cancel_cost, 0)
    data["PotentialRevenue"] = (data.RevenuebyStays
                                - data.RevenueLossbyUpgrade
                                - data.RevenueLossbyCancel)
    return data


def fit_probability_model(data, features, target, seed=42):
    """Fit a Random Forest with an out-of-sample evaluation split."""
    x, y = data[features], data[target]
    categorical = x.select_dtypes(include=["object", "category"]).columns
    numeric = x.columns.difference(categorical)
    prep = ColumnTransformer([
        ("category", OneHotEncoder(handle_unknown="ignore"), categorical),
        ("numeric", "passthrough", numeric)])
    model = Pipeline([
        ("preprocessing", prep),
        ("classifier", RandomForestClassifier(
            n_estimators=400, min_samples_leaf=3,
            class_weight="balanced_subsample", random_state=seed, n_jobs=-1))])
    x_train, x_test, y_train, y_test = train_test_split(
        x, y, test_size=.20, stratify=y, random_state=seed)
    model.fit(x_train, y_train)
    return model, model.score(x_test, y_test)


def backtest_premiums(data, upgrade_premium, cancel_premium):
    """Apply refundable premiums and test the paired revenue improvement."""
    tested = data.copy()
    upgrade_recovery = (upgrade_premium * tested.Nights).clip(
        upper=tested.RevenueLossbyUpgrade)
    cancel_recovery = (cancel_premium * tested.Nights).clip(
        upper=tested.RevenueLossbyCancel)
    tested["RevenueWithBothStrategies"] = (
        tested.PotentialRevenue + upgrade_recovery + cancel_recovery)
    test = ttest_rel(tested.RevenueWithBothStrategies,
                     tested.PotentialRevenue, alternative="greater")
    return tested, {"revenue": tested.RevenueWithBothStrategies.sum(),
                    "t_statistic": test.statistic, "p_value": test.pvalue}
```

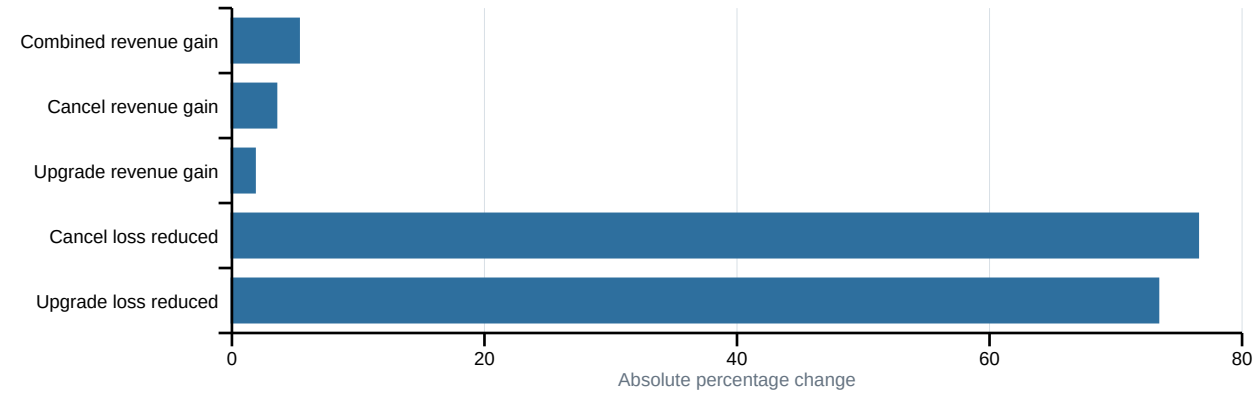
Back-test Results & Business Impact

89,634	28	2	+5.28%
CLEAN BOOKINGS	MODEL FEATURES	RANDOM FORESTS	COMBINED REVENUE

Reported project outcomes

OUTCOME	BASELINE	STRATEGY	CHANGE	P-VALUE
Upgrade loss	€637,988	€170,090	**-73.34%**	8.39e-287
Cancellation loss	€1,190,536	€279,872	**-76.49%**	9.04e-267
Upgrade strategy revenue	€26,083,973	€26,551,871	**+1.79%**	3.04e-06
Cancellation strategy revenue	€26,083,973	€26,994,636	**+3.49%**	1.30e-18
Combined-strategy revenue	€26,083,973	€27,462,534	**+5.28%**	1.00e-40

Effect sizes from historical back-testing



INTERPRETATION

The models convert booking-level probabilities into expected-loss premiums. Historical back-tests indicate that the upgrade and cancellation strategies reduce their respective losses by 73.34% and 76.49%; together they raise potential revenue from €26.08M to €27.46M. These are results from the submitted team project, not a live randomized experiment.